

**Escrevendo software de
forma eficiente com IA**

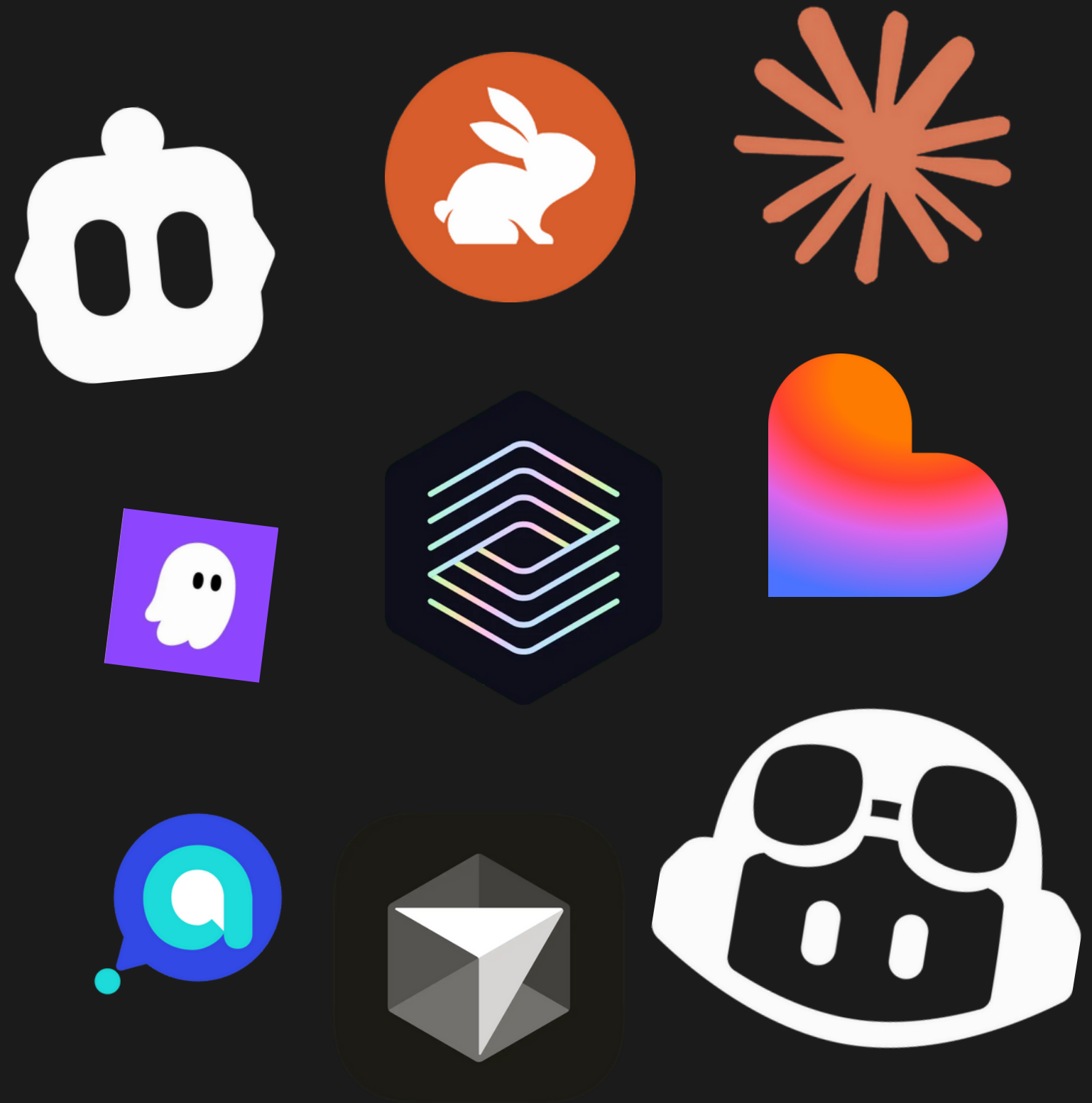
Romulo Assis

- 👤 Desde 2014
- 💻 Engenheiro de Software @iFood
- 📖 Pós graduado em IA
- 🐦 Especializando em AI Engineering



Panorama geral

- Code Agents
- Modelos treinados para codificação
- Hype da IA
- Demissões em massa com promessa de automação



Da empolgação ...

STACK OVERFLOW 2024

76% dos devs já usam ou planejam usar ferramentas de IA no desenvolvimento

GITHUB COM A ACCENTURE

Houve +8,7% de PRs enviados, +15% em PRs aprovados e +84% em builds bem-sucedidos

ROBINHOOD

Cerca de 50% do novo código já é gerado por IA.

...a frustração

Quem nunca disse:

“a IA não funciona direito”
“o código vem todo bugado”



...e qual o motivo disso?

VIBE CODING

O que é?

'Vibe coding' é a palavra do ano de 2025 de acordo com dicionário britânico; entenda o significado

Termo se popularizou em fevereiro e foge dos métodos tradicionais de programação

Por: Alice Labate

6 nov 2025 - 12h16

Compartilhar ➞

[Exibir comentários](#)

Na **informática**, o **vibe coding** é uma técnica de **programação** orientada por **Inteligência Artificial**; onde a usuário descreve um problema ou um cenário em poucas frases na forma de um **prompt** para um **modelo de linguagem grande (LLM)**, então o modelo gera um **código-fonte** de um **software**. Assim o programador não faz a codificação manual e passa a testar e refinar este código gerado pela IA.^{[1][2][3]}

Os defensores desta técnica afirmam que ela permite que programadores **hobbistas** produzam software sem o treinamento extensivo e as habilidades anteriormente necessárias para a **engenharia de software**.^[4]

Sinais que estou em Vibe Coding

PROMPT GIGANTES

Querer passar tudo que deve ser feito de uma única vez

ASSUMIR O PAPEL PASSIVO

Só assistir o modelo trabalhar sem entender ou intervir quando preciso

NÃO ENTENDER FUNDAMENTOS

Quem mais defende vibe coding, não tem os fundamentos de engenharia de software

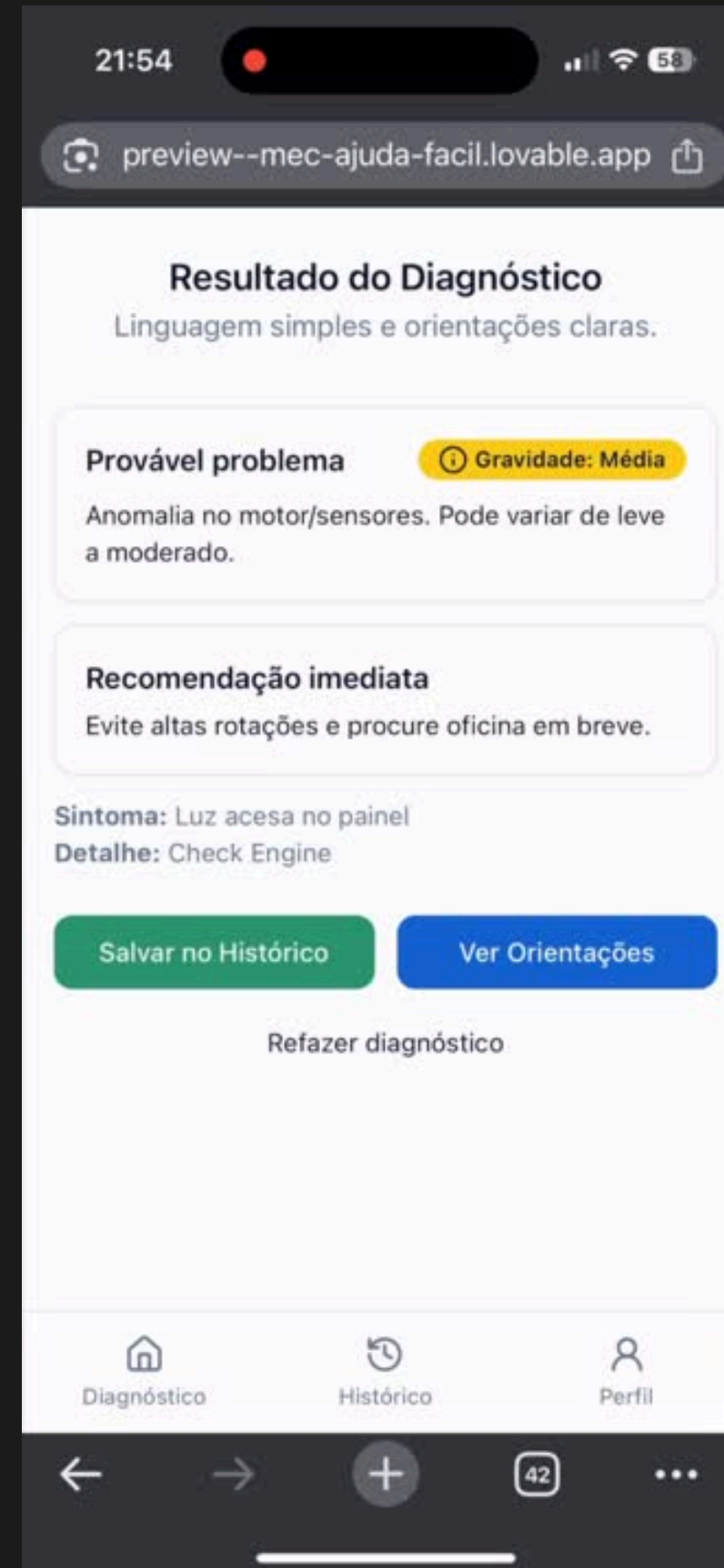
Onde usar?

Explorar ideias rapidamente (spikes, POCs, playground)

Gerar exemplos de código para estudo
“como seria um useCase em Nest com TDD?”

Criar protótipos descartáveis para entender uma abordagem

Onde usar?



Onde NÃO usar?

Features críticas de negócio (pagamento, segurança, compliance)

Evolução de sistemas legados sensíveis

Decisões de arquitetura, modelagem de domínio e integrações complexas

Problemas principais

Código desalinhado com as regras de negócio

Incompatibilidade com a arquitetura, stack e constraints reais

Falta de rastreabilidade: ninguém sabe de onde veio a decisão

Débito de conhecimento: dev não aprende, só executa

Impacto para o time e o produto



**...existe uma forma
melhor de usar IA?**

Desenvolvimento guiado por IA

Conceito

Desenvolvedor é o dono das decisões

Papel da IA: par programador + assistente
de contexto + acelerador

Ganho de performance com segurança e
confiabilidade

Papéis bem definidos

Desenvolvedor: entende o problema, define o plano, valida e decide

IA: ajuda a detalhar tarefas, sugerir implementações, encontrar edge cases, escrever e refinar código

Case

AI-Driven Testing / iFood

DESAFIO

Meta: +60% coverage em 1 mês
-10% coverage, 45 módulos, +20K linhas de código.

TESTES GERADOS

149 arquivos, +28K linhas criadas, +2K casos de testes, 62% coverage

CASO DE SUCESSO

Módulo de KDS, 0% a 98,8% em 3 horas
(25 arquivos, 1.638 linhas de teste)

Como escrever software de forma
eficiente com IA?

Regras de ouro

→ Mantenha um setup

MCP

Code Review no próprio agent

Tenha um workflow bem definido

Regras de ouro

Mantenha um setup

→ MCP

Code Review no próprio agent

Tenha um workflow bem definido

Regras de ouro

Mantenha um setup

MCP

→ Code Review no próprio agent

Tenha um workflow bem definido

Regras de ouro

Mantenha um setup

MCP

Code Review no próprio agent

→ Tenha um **workflow** bem definido

WORKFLOW

Workflow

→ Contexto do projeto

Planejamento de tarefas

Plano de execução

Execução do plano

Atualização dos contextos

Workflow

Contexto do projeto

→ Planejamento de tarefas

Plano de execução

Execução do plano

Atualização dos contextos

Workflow

Contexto do projeto

Planejamento de tarefas

→ Plano de execução

Execução do plano

Atualização dos contextos

Workflow

Contexto do projeto

Planejamento de tarefas

Plano de execução

→ Execução do plano

Atualização dos contextos

Workflow

Contexto do projeto

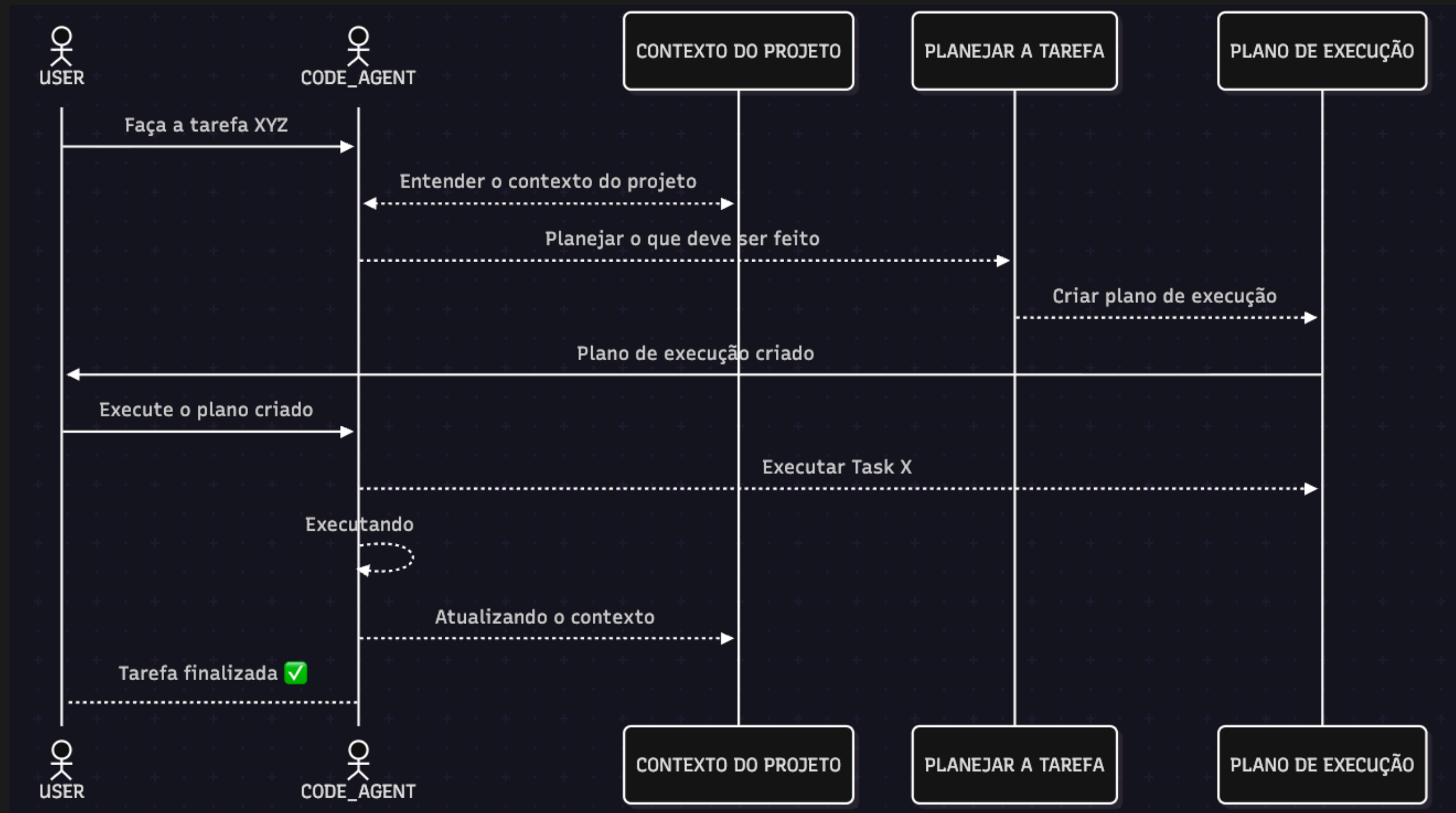
Planejamento de tarefas

Plano de execução

Execução do plano

→ Atualização dos contextos

Workflow



Bora codar...



Encurte seus links de forma simples

OLLO.li é o encurtador de links que você precisa. Rápido, simples e confiável.

Começar Agora



Simples e Rápido

Cole seu link e receba um link curto instantaneamente. Sem complicações.



Redirecionamento Rápido

Seus links encurtados redirecionam instantaneamente para o destino original.



Confiável

Seus dados e informações estão seguros conosco. Conte com a OLLO.li.

OLLO.li

https://meusitegigante.com.br/home



https://ollo.li/j978jom

Contexto do Projeto

Contexto do projeto

Usando o [Codex](#) com o comando `/init` ele gera o [AGENTS.md](#)

Code Agente sempre olhará para esse arquivo antes de qualquer tarefa

Colocar o máximo de informações sobre seu projeto

Criar um links simbólico para outros Code Agents: [CLAUDE.md](#)

para agentes e humanos.

Project Structure & Module Organization

Fastify + TypeScript live under `src/` and are split by modules mirroring OLLO.li features: `modules/auth` (JWT/session lifecycle), `modules/user` (sign-up, profile, password), `modules/link` (shortener CRUD + caching), `modules/email` (verification + recovery templates) e `modules/upload` (Supabase storage). Cross-cutting pieces stay in `configurations/` (logger, decorators, rate limit, Swagger), `infra/` (Prisma, Redis, Supabase clients/storages) e `helpers/` (hashing, HTTP codes, TTL helpers). Tests accompany each module (`__tests__`) with shared harnesses under `src/tests/`. Docs, diagrams e templates residem em `docs/` e `resource/`.

Build, Test, and Development Commands

- `make setup`: onboarding completo (nvm use, `npm install`, sobe `docker-compose` com Postgres/Redis, roda `prisma generate`, `prisma:db:push` e `npm run seed`).
- `npm run dev`: levanta Fastify com Nodemon/inspector, carregando `.env.development` e exibindo logs coloridos se `DEBUG_MODE=true`.
- `npm run docker`: inicia somente a stack de infra local (containers `ollo-li-postgres` e `ollo-li-redis`).
- `npm run build` / `npm run start`: compila via `tsc` + `tsc-alias` e executa `dist/src/server.js` em produção.
- `npm run test`, `test:watch`, `test:ci`: executam Jest com cobertura (mínimo 80% para branches/functions/lines/statements).

Coding Style & Naming Conventions

Utilize TypeScript estrito, path alias `@/` e sufixos por responsabilidade (`*.controller.ts`, `*.service.ts`, `*.schema.ts`). Controllers tratam Fastify requests/replies, services concentram regra de negócio com `Context { prisma }`, schemas usam Zod e expõem tipos. Enums/mensagens seguem `UPPER_SNAKE_CASE`. Antes de abrir PR, rode `npm run lint` (ESLint + `standard-with-typescript`) e mantenha funções curtas, com erros centralizados em `helpers/error-handler.ts`.

Testing Guidelines

Jest (`ts-jest`) cobre unidades e integrações. Nomeie arquivos como `link.service.unit.test.ts` ou `auth.routes.integration.test.ts`. Use mocks globais em `src/tests/` (`prisma.ts`, `redis.ts`, `app.ts`) para simular banco/cache e injetar cookies JWT (`tests/jwt.ts`). Não reduza cobertura abaixo de 80%; explore `npm run test:watch` `src/modules/link/__tests__/link-integration.test.ts` para iterações rápidas.

Commit & Pull Request Guidelines

Mensagens seguem Conventional Commits e são auxiliadas por `npm run commit` (Commitizen). Exemplos: `feat(link): add metadata validation` ou `fix(auth): refresh cookie flags`. Pull requests devem referenciar issues (`Closes #123`), resumir mudanças, listar comandos/testes executados e anexar evidências relevantes (p. ex., resposta do endpoint, novo fluxo em *Mermoid*). Certifique-se de que `npm run build` e `npm run test` passem antes de revisar.

Problemas

Projetos grandes e com muitas regras,
contexto fica muito grande

Gasto excessivo de tokens para
entendimento básico do contexto

Pode causar alucinações por conta da
necessidade de *resumir todo* o contexto

Expansão do contexto

Memory Bank

| Memory Bank

```
<context>
Você é um(a) Arquiteto(a) de Software Senior responsável por analisar profundamente este projeto e produzir documentação enxuta, clara e útil para pessoas e modelos de IA.
</context>
<task>
Crie um diretório chamado <code>ai-context</code> que funcionará como um banco de memória do projeto. Dentro dele, crie arquivos em Markdown cobrindo os aspectos principais e um índice que permita à IA e aos humanos encontrarem rapidamente o que precisam.
</task>

<aspects>
- arquitetura
- regras de negócio
- testes/qualidade
- infraestrutura
- deploy/ci-cd
- observabilidade (logs/métricas/traces)
- segurança/compliance (sem expor segredos)
- glossário e referências
</aspects>

<additional_rules>
<rule>Todos os arquivos devem ser Markdown (.md) com linguagem direta e exemplos quando necessário.
<rule>Inclua fluxogramas em Mermaid para os pontos críticos
(pelo menos um flowchart end-to-end e um sequenceDiagram de um caso de uso chave).</rule>
<rule>Use links relativos entre os arquivos para navegação.</rule>
<rule>Não incluir segredos/credenciais; use placeholders quando necessário.</rule>
</additional_rules>
```

Contexto profundo do projeto

Expansão detalhada e profunda de cada aspecto do projeto

Unificar tudo em uma pasta
[/ai-context](#)

Cada arquivo um tema
(arquitetura, testes, CI/CD, etc)

Linkar com AGENTS.md

EXPLORER

OLLO-LINK-API

.github

.vscode

AI

ai-context

arquitetura.md

deploy-ci-cd.md

glossario-referencias.md

index.md

infraestrutura.md

observabilidade.md

regras-de-negocio.md

seguranca-compliance.md

testes-qualidade.md

CODEXPERIENCE

coverage

docs

prisma

resource

scripts

src

.dockerignore

.env.development

.env.example

.env.test

.eslinttrc.js

.gitignore

.nvmrc

.release-it.json

AGENTS.md

changelog.config.js

CHANGELOG.md

CLAUDE.md

codecov.yml

docker-compose.vml

OUTLINE

TIMELINE

arquitetura.md

Preview arquitetura.md

Decorators

Rate limit & Errors

Router

CrossCutting

JWT + Cookies

Cross-cutting

*.route.ts

Controller

Service com Context

ORM

Cache

Storage

Email

PostgreSQL via Prisma

Redis TTL

Supabase Storage

Mailersend

- O fluxo acima representa uma chamada típica autenticada (ex.: criação de link). Controllers usam `ErrorHandler` para normalizar respostas HTTP (`src/helpers/error-handler.ts`).
- Serviços podem encadear Prisma + Redis (cache de links) ou Supabase (upload) antes de retornar ao controller.

Convenções de integração

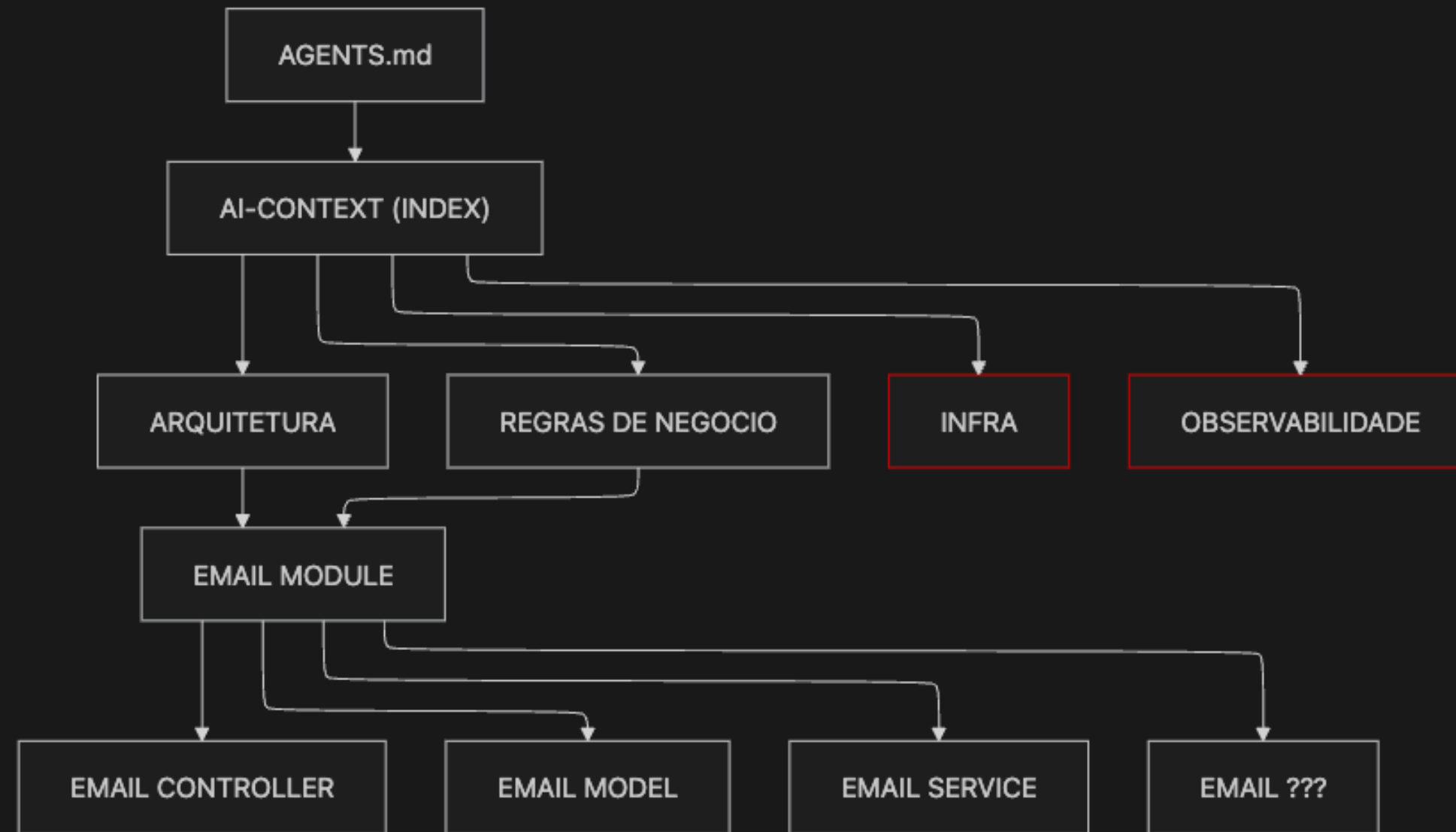
- Alias `@/`: configurado no `tsconfig.json` e reconhecido via `tsconfig-paths` e Jest (`moduleNameMapper`).
- Contexto único: controllers instanciam `context: { prisma }` importado de `@/infra` para facilitar testes com mocks.
- Responsabilidade única: regras de negócio ficam em `services`; controllers apenas orquestram dependências e formatam resposta.
- Documentação cruzada: decisões de domínio detalhadas em [Regras de Negócio](#); integrações operacionais em [Infraestrutura](#).

feat/codexperience

Launchpad

0 0 10

Go Live



Quero entender como funciona o envio de email da aplicação

Tenha um roteador

- Somente o contexto necessário é buscado
- Modelo “*indexa*” a informação mais rapidamente
- Economia de tokens e \$\$\$ 💰💰

Banco de Contexto da OLLO.li API

Este diretório concentra resumos curtos sobre arquitetura, processos e regras-chave da API.

Use-o como ponto de partida para humanos e assistentes avaliarem impacto de mudanças sem vasculhar todo o repositório.

Navegação rápida

- [Arquitetura](./arquitetura.md) – visão do stack Fastify/TypeScript, módulos e fluxo end-to-end.
- [Regras de Negócio](./regras-de-negocio.md) – policies de autenticação, links, emails e uploads com sequência detalhada.
- [Testes & Qualidade](./testes-qualidade.md) – convenções de Jest, cobertura mínima e lint.
- [Infraestrutura](./infraestrutura.md) – bancos, cache, storage e scripts de provisionamento.
- [Deploy & CI/CD](./deploy-ci-cd.md) – pipelines GitHub Actions, release e gatilho de deploy.
- [Observabilidade](./observabilidade.md) – logging, rate limit, healthcheck e lacunas de métricas.
- [Segurança & Compliance](./seguranca-compliance.md) – proteções de dados, tokens, CORS e segredos.
- [Glossário & Referências](./glossario-referencias.md) – termos frequentes e links úteis.

> Todos os arquivos usam apenas referências relativas e evitam expor segredo; substitua por placeholders quando necessário.

Implementação

O que precisamos de saber
antes de nos aventurar a
"*codar*" guiados à IA?

MEMÓRIA

Uso de memória de longo prazo para não se perder durante tarefas futuras.

PLAN E ACT MODE

É extremamente eficiente e recomendado pedir para o modelo planejar a tarefa antes de executar.

TDD

Com TDD os modelos dão garantia que “pensaram” como implementar antes de fazer.



Arcadio Buendia

Acme Corp



Demanda de Produto – Rota de Redirecionamento de Links

Queremos fechar o ciclo do nosso encurtador de links: hoje já é possível gerar um link curto, mas ainda não existe uma rota pública padrão para quando alguém clica nesse link.

Precisamos de uma rota única que receba o link encurtado e leve a pessoa automaticamente para a URL original, ou avise claramente quando o link não existir.

O que esperamos ter ao final:

- Um endereço público no formato `https://NOSSO_DOMINIO/r/{hash}` que possa ser usado em campanhas, e-mails, etc.
- Se o hash existir, deve ser redirecionado para a URL original com o status de redirecionamento correto.
- Se o hash não existir, deve receber uma resposta de “não encontrado” (404).

Context

Você é um engenheiro de software senior, você deve executar a tarefa de criar uma rota para redirecionar as urls encurtadas

Tasks

Task 1

Criar módulo para fazer o redirecionamento do LINK, seguindo as regras:

- O novo módulo deve ter o nome de ``redirector``
- Deve seguir a arquitetura presente no projeto
- A rota de redirecionamento deve ter um prefixo: ``r`` de ``redirect`` com parametro que é o hash encurtado, ex: `http://MINHA_URL/r/HASH`

Task 2

O módulo deve ter o seguinte comportamento:

- O usuário deve chamar a rota encurtada por um método GET,
- Se o link encurtador existir deve retornar um 301 e redirecionar para a rota original, ex: `http://localhost:3000/r/ABAIJD` → `https://www.meuoriginalsite.com`
- Se a rota não existir, deve retornar um 404
- Os testes devem bater no mínimo 90% de coverage

Task 3

- Use o MCP do Postman para criar essa endpoint dentro do Workspace OLLO e Collection Ollo.li, toda as rotas estão exportadas no arquivo docs/postman_collection.json
- Adicione no docs/postman_collection.json essa nova rota
- Usando o MCP do Postman teste se a rota está funcionando seguindo o fluxo:
LOGIN → ENCURTAR UM LINK → REDIRECIONAR
Se retornar 301 sucesso, se retornar 404 falha na implementação

Planejar o que fazer

TAREFAS

Quebre em várias tarefas, explique detalhadamente o que cada uma deve fazer

PLAN MODE

Use o modo de PLAN mode do Code Agent, ele usará o máximo de tokens para refinar o que fazer

TO-DO LIST

Peça para o modelo executar as tarefas no modelo de TO-DO list

Task 2

O módulo deve ter o seguinte comportamento:

- O usuário deve chamar a rota encurtada por um método GET,
- Se o link encurtador existir deve retornar um 301 e redirecionar para a rota original, ex: http://localhost:3000/r/ABAIJD → https://www.meuoriginalsite.com
- Se a rota não existir, deve retornar um 404
- Os testes devem bater no mínimo 90% de coverage

Task 3

- Use o MCP do Postman para criar essa endpoint dentro do Workspace OLLO e Collection Ollo.li, toda as rotas estão exportadas no arquivo docs/postman_collection.json
- Adicione no docs/postman_collection.json essa nova rota
- Usando o MCP do Postman teste se a rota está funcionando seguindo o fluxo:
LOGIN → ENCURTAR UM LINK → REDIRECIONAR
Se retornar 301 sucesso, se retornar 404 falha na implementação

Task 4

Atualize no ``ai-context`` se necessário o novo comportamento e regras de negócio

Additional Rules

- Crie um plano para a tarefa em ``AI/PLANS/REDIRECTOR_PLAN.md`` usando o template ``AI/PROMPTS/TEMPLATES/TEMPLATE_EXECUTION_PLAN.md`` como base
- Siga todas as regras de forma restritas ditas no plano criado
- Ao final de cada tarefa, coloque o check  na tarefa finalizada no plano ``AI/PLANS/REDIRECTOR_PLAN.md``
- Sempre use o plano ``AI/PLANS/REDIRECTOR_PLAN.md`` como memória de longo prazo para saber qual tarefa foi feita, qual está sendo feita e qual será a próxima
- Apenas inicie uma tarefa após finalizar a outra
- Sempre rode todos os testes para ver se houve algum efeito colateral em outra parte que já está funcionando, se sim, resolva

Planejar o que fazer

PROMPT ENGINEERING

Chain-of-Thought, ReAct, Few-Shot, etc

PLANO DE EXECUÇÃO

Formato markdown, salve e commit esse plano durante a tarefa

CONTEXTO

Atualizar o *memory bank*, assim o projeto sempre terá o contexto atualizado

EXPLOLER

OLLO-LINK-API

> .github

> .vscode

> AI

> ai-context

CODEXPER...

PROMPTS

TEMPLA...

TEMPL...

EXPAND...

PROMPT...

coverage

docs

prisma

resource

images

logo-ollo.png

scripts

src

.dockerignore

.env.development

.env.example

.env.test

.eslintrc.js

.gitignore

.nvmrc

.release-it.json

AGENTS.md

OUTLINE

TIMELINE

PROBLEMS 6

OUTPUT

TERMINAL

PORTS

GITLENS

SPELL CHECKER 6

DEBUG CONSOLE

codex-x86_64-app

ollo-link-api via base took 40m 12,1s ...

→

ollo-link-api on feat/codexperience [\$!?] is 1.2.9 via v20.18.1 on v28.3.2 runs 00 vi ...

> _ OpenAI Codex (v0.58.0)

model: gpt-5.1-codex medium /model to change

directory: ~/www/ollo/ollo-link-api

To get started, describe a task or try one of these commands:

/init - create an AGENTS.md file with instructions for Codex

/status - show current session configuration

/approvals - choose what Codex can do without approval

/model - choose what model and reasoning effort to use

/review - review any changes and find issues

> execute o prompt CODEXPERIENCE/PROMPTS/PROMPT_TASK.md, não execute nada apenas crie primeiro o plano.

• Working (0s • esc to interrupt)

> S

100% context left

Ln 43, Col 127 (7 selected)

Spaces: 2

UTF-8

LF

{ } Markdown

Port : 5500

Prettier

Stage 0 – Planejamento & Cenários

- [x]  ****Test Fails**** – Mapear rotas e cenários esperados na suite (301, 404) e preparar testes para falhar confirmando ausência de ``/r/:hash``.
- [x]  ****Code**** – Criar esqueleto do módulo ``redirector`` (controller/service/schema/routes) alinhado à arquitetura antes de implementar lógica.
- [x]  ****Green**** – Garantir que lint/types passam e nova suite evidencia falhas planejadas, servindo como baseline.
- [x]  ****Notas**** – Reuso fará lookup em cache/link service; mocks extras necessários (``bcrypt``, ``@/infra/clients/prisma``).

Stage 1 – Implementação Redirector

- [x]  ****Test Fails**** – Reexecutar a suite adicionada no Stage 0 para evidenciar falhas antes de escrever código definitivo (prova de TDD).
- [x]  ****Code**** – Implementar serviço de lookup, controller, rota Fastify (``/r/:hash``), compor com link repository/TTL helpers, adicionar erros adequados.
- [x]  ****Green**** – Rodar ``npm run test`` (ou subset) garantindo $\geq 90\%$ coverage no módulo e corrigir regressões.
- [x]  ****Notas**** – ``resolveRedirectDestination`` usa cache + Prisma; redirect responde 301 com ``Cache-Control: no-store``; cobertura confirmada via ``coverage-redirector/`` (routes/service > 95%).

Stage 2 – Postman & Fluxo End-to-End

- [x]  ****Test Fails**** – Atualizar ``docs/postman_collection.json`` adicionando rota e preparar execução MCP Postman esperando 404 antes de publicar.
- [x]  ****Code**** – Criar/atualizar request na Collection OLLO via MCP (Workspace OLLO, Collection Ollo.li), sincronizar JSON local.
- [x]  ****Green**** – Executar fluxo LOGIN→ENCURTAR→REDIRECIONAR com MCP Postman; considerar sucesso apenas com 301.

Plano de execução

TDD

Memória de longo prazo

Tarefas definidas

Possibilidade de checkpoint

Possível retirar métricas

Executar o plano

SOURCE CONTROL

CHANGES

Message (⌘Enter to ...)

Commit

Changes 14

AI/PLANS

REDIRECTOR_PLA... U

src

helpers

http.ts M

modules

redirector

__mocks__

redirect-link.ts U

__tests__

redirector.inte... U

redirector.serv... U

schemas

index.ts U

redirector.sch... U

services

index.ts U

redirector.serv... U

redirector.route.... U

upload / __tests__

upload-unit.tes... M

tests

prisma.ts M

GRAPH

GITLENS

PROBLEMS OUTPUT TERMINAL PORTS GITLENS SPELL CHECKER DEBUG CONSOLE

codex-x86_64-app

58 -| `Stage 1` | `21:49` | `pending` | `Novos testes para Cache-Control e revalidação de cache quebram com comportamento atual.` |

58 +| `Stage 1` | `21:49` | `ok` | `Suíte completa (npm run test) passando; redirector com 100% coverage após cenários extra.` |

59 | `Stage 2` | `HH:MM` | `` | `` |

- I'm mapping out Stage 2 tasks focused on using Postman via MCP, including managing the "OLLO" workspace and "Ollo.li" collection. I'll need to gather workspace and collection IDs, inspect the docs/postman_collection.json, and plan how to update Postman collections through the various mcp_postman_ functions, while interpreting the instructions carefully. Let's explore the repo for relevant Postman details next!
- Ran rg -n "Ollo\\.li" -n docs -g "*.json"
 - (no output)
- Explored
 - Read postman_collection.json
 - List docs
- Ran rg -n "postman_collection" -g "*.json" -n ..
 - (no output)
- Explored
 - Search Ollo.li
 - Search *postman* in .
- Searching for Postman collection file (37m 58s • esc to interrupt)

explain this codebase

49% context left • ? for shortcuts

testando...

OLLO New Import

Collections

Environments

History

Flows

Files

Files BETA

Search collections

OLLO.li

Auth

POST Login

POST Refresh Token

Users

Email

Link

POST Shortener a link

GET Get All links

PUT Update a link

GET Redirect

GET Chek API is UP

GET Resolve Redirect

POST Login

POST Shortener a link

OLLO.li / Link / Shortener a link

Save Share

POST

{{base_url}} /links/shortener

Send

Docs Params Auth Headers (12) Body Scripts Tests Settings

raw

JSON

Schema

```
1 {
2   "url": "https://codexperience.com.br/",
3   "validAt": "2025-12-31T19:52:21.208Z",
4   "metadata": {
5     "title": "codeXperience",
6     "description": "Maior evento do brasil"
7   }
8 }
```

Response History

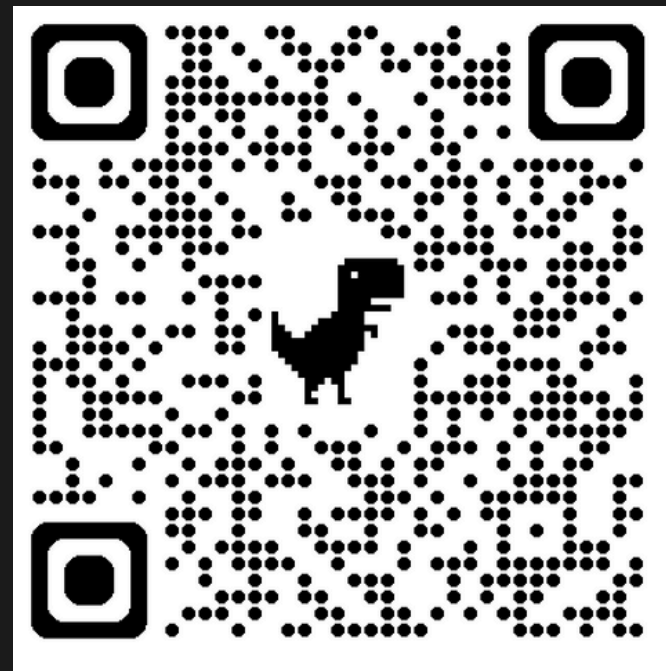


Click Send to get a response

Tarefa entregue com sucesso



Obrigado!



 @assis.coder

 romulo-assis

FONTES

https://survey.stackoverflow.co/2024/ai?utm_source=chatgpt.com

https://www.businessinsider.com/robinhood-ceo-majority-new-code-ai-generated-engineer-adoption-2025-7?utm_source=chatgpt.com

https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-in-the-enterprise-with-accenture/?utm_source=chatgpt.com

<https://www.tecmundo.com.br/seguranca/406906-sapphos-tinder-para-mulheres-tem-falha-grave-denunciada-por-usuario-e-sai-do-ar.htm>

https://pt.wikipedia.org/wiki/Vibe_coding

<https://startups.com.br/negocios/inteligencia-artificial/como-o-vibe-coding-se-tornou-xodo-dos-vcs-e-um-problema-de-seguranca/>

https://www-nbcnews-com.translate.goog/tech/social-media/tea-app-hacked-13000-photos-leaked-4chan-call-action-rcna221139?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc&_x_tr_hist=true